

Live Chat

Introduction

Souvenez-vous : en 1999, on faisait "toctoc" sur une messagerie pour dire qu'on était là. Aujourd'hui, les équipes vivent sur d'autres outils : serveurs, salons texte et vocaux, visio, partage d'écran, le tout dans une seule application qui tourne dans le navigateur.

Vous êtes développeurs dans une startup spécialisée dans les outils de communication d'entreprise. Votre client historique, RetroComm, possède déjà une petite messagerie interne façon MSN (la v1, que vous reprenez). Le problème : leurs équipes jonglent entre le chat, un outil de visio tiers et un partage d'écran qui ne marche jamais. Le directeur technique veut une seule plateforme unifiée, façon Discord, hébergée en interne.

L'objectif : faire évoluer le produit vers une application temps réel complète mêlant :

- de la messagerie instantanée (texte) via WebSockets,
- des appels audio/vidéo et un partage d'écran en pair-à-pair (WebRTC),
- une organisation en serveurs et salons (texte + vocaux),
- une architecture front React / back Node avec une persistance *SQL et NoSQL*.



« 💡 Pourquoi deux technologies temps réel ? Le texte transite par le serveur via WebSocket (Socket.io). Le média audio/vidéo voyage en pair-à-pair directement entre navigateurs (WebRTC) : le serveur ne sert qu'à la signalisation (mise en relation des pairs). Comprendre cette séparation est le cœur technique du projet. »

Contraintes Client

RetroComm est une agence de communication d'une cinquantaine de collaborateurs répartis sur plusieurs sites. Leur messagerie interne v1 (texte uniquement) ne suffit plus : les équipes veulent se parler et se voir sans quitter l'outil.

Le directeur technique commande une v2 capable de :

- Organiser les échanges par serveurs d'équipe contenant des salons thématiques.
- Discuter en temps réel (texte) ET passer des appels audio/vidéo (1-to-1 et petits groupes).
- Partager son écran pour les revues et démos.



Fonctionnalités Attendues

Module d'authentification

Fonctionnalité	Description
Inscription	Création de compte : pseudo unique, email, mot de passe
Connexion	Authentification sécurisée par JWT
Profil utilisateur	Pseudo, avatar, statut personnalisé (clin d'œil MSN)
Déconnexion	Fermeture de session propre

Module présence (Temps Réel)

Fonctionnalité	Description
Liste des connectés	Affichage temps réel des utilisateurs en ligne
Status de présence	En ligne, Absent, Occupé, Hors ligne
Mise à jour instantanée	Détection automatique connexion/déconnexion
Etat Stocké côté Redis	La présence est gérée via Redis (voir partie données)

Module serveurs & Salons (Le coeur)

Fonctionnalité	Description
Création de serveur	Nom, icône, propriétaire (admin)
Invitation / appartenance	Code d'invitation ou ajout de membres



Rôles basiques	Admin (gère le serveur/salons) vs Membre
Salons texte	Plusieurs salons de discussion écrite par serveur
Salons vocaux	Salons dédiés aux appels audio/vidéo de groupe
Liste des membres	Voir qui appartient au serveur et qui est connecté

Module messagerie (Temps Réel + historique)

Fonctionnalité	Description
Messages de salon	Envoi/réception temps réel via WebSocket
Messages privés (DM)	Conversation 1-to-1 entre deux utilisateurs
Historique persisté	Stockage et chargement des messages depuis MongoDB
Horodatage	Date/heure de chaque message
Indicateur de frappe	« X est en train d'écrire... »
Notifications non-lus	Indicateur de nouveaux messages

Module Audio/Vidéo (WebRTC)

Fonctionnalité	Niveau	Description
Appel 1-to-1 audio/vidéo	Obligatoire	Appeler un utilisateur en privé, le voir et l'entendre
Salon vocal de groupe	Obligatoire	Appel de groupe en mesh jusqu'à 3 participants



Contrôles d'appel	Obligatoire	Couper/activer micro, couper/activer caméra, raccrocher
Indicateur « en appel »	Obligatoire	Voir qui est dans un salon vocal en temps réel
Partage d'écran	Bonus	Diffuser son écran via getDisplayMedia

⚠ Le mesh : dans un appel de groupe en mesh, chaque participant ouvre une connexion vers tous les autres. À 3 personnes, cela fait 3 connexions par participant. C'est pour cela que la limite est fixée à 3 : au-delà, il faudrait une architecture SFU

Interface utilisateur

Fonctionnalité	Description
Barre serveurs	Liste des serveurs de l'utilisateur (icônes)
Barre salons	Salons texte/vocaux du serveur sélectionné
Zone de chat	Messages avec scroll, pseudo, avatar, horodatage
Zone de saisie	Input + envoi (Entrée), indicateur de frappe
Panneau d'appel	Vignettes vidéo des participants + contrôles
Liste des membres	Membres du serveur + statut de présence



Spécifications Techniques

Stack Obligatoire

Composant	Technologie
Frontend	React (Vite) + state management (Context API ou Zustand)
Temps réel (texte/présence/signalisation)	Socket.io
Média audio/vidéo	WebRTC
Backend	Node.js + Express
Base relationnelle	PostgreSQL ou MySQL
ORM	Prisma
Base documentaire	MongoDB (historique des messages)
Cache clé-valeur	Redis (présence + état des salons vocaux)
Authentification	JWT (jsonwebtoken) + bcrypt
Conteneurisation	Docker + Docker Compose
Intégration continue	GitHub Actions (ou GitLab CI)

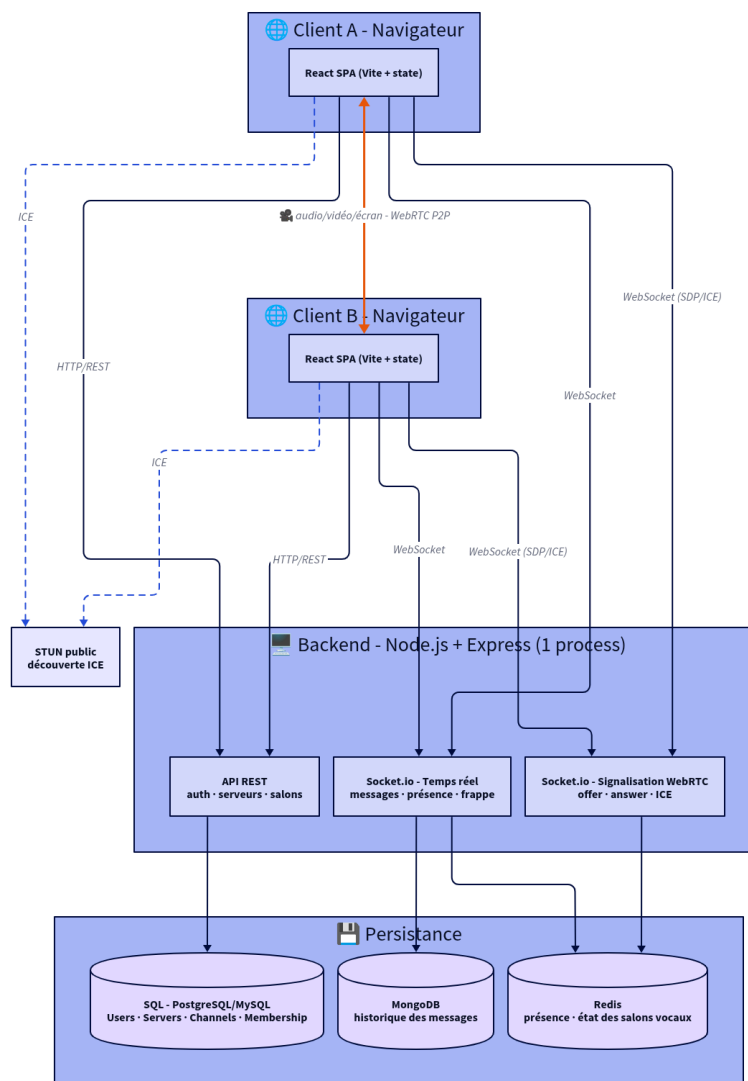
Stack Obligatoire

Donnée	Base	Pourquoi
Comptes, serveurs, salons, appartenances, rôles	SQL	Données structurées et relationnelles (jointures, intégrité référentielle)



Historique des messages	MongoDB	Gros volume, schéma flexible (réactions, pièces jointes, embeds), lecture par salon
Présence (qui est en ligne) + état des salons vocaux	Redis	Données éphémères, ultra-rapides, mises à jour en continu

Architecture cible





Communication WebSocket (Socket.io)

Texte & présence :

Événement	Direction	Description
connection / disconnect	Client $\rightleftharpoons$ Serveur	Connexion / déconnexion
join-channel / leave-channel	Client $\rightarrow$ Serveur	Rejoindre / quitter un salon texte
send-message	Client $\rightarrow$ Serveur	Envoi d'un message
receive-message	Serveur $\rightarrow$ Clients	Diffusion d'un message
presence-update	Serveur $\rightarrow$ Clients	Changement de statut/présence
typing / user-typing	Client $\rightleftharpoons$ Serveur	Indicateur de frappe

Signalisation WebRTC :

Événement	Direction	Description
join-voice / leave-voice	Client $\rightarrow$ Serveur	Rejoindre / quitter un salon vocal
voice-participants	Serveur $\rightarrow$ Clients	Liste à jour des participants
webrtc-offer	Client $\rightarrow$ Serveur $\rightarrow$ Client	Offre SDP vers un pair
webrtc-answer	Client $\rightarrow$ Serveur $\rightarrow$ Client	Réponse SDP au pair
ice-candidate	Client $\rightarrow$ Serveur $\rightarrow$ Client	Échange des candidats ICE
screen-share (bonus)	Client $\rightarrow$ Clients	Démarrage/arrêt du partage d'écran



Flux WebRTC (à comprendre et documenter)

1. L'utilisateur rejoint un salon vocal → `getUserMedia()` récupère caméra + micro.
2. Pour chaque autre participant : création d'une `RTCPeerConnection`.
3. Le pair qui initie crée une offre SDP → envoyée via `Socket.io` (`webrtc-offer`).
4. Le pair distant répond par une `answer SDP` (`webrtc-answer`).
5. Les deux échangent leurs ICE candidates (`ice-candidate`) pour trouver le meilleur chemin réseau.
6. La connexion P2P s'établit, l'événement `ontrack` permet d'afficher le flux distant.
7. À 3 participants, chacun maintient 2 connexions → mesh complet. « Un serveur STUN public (ex. `stun:stun.l.google.com:19302`) suffit en développement local/LAN. Un serveur TURN n'est pas exigé. »

Modèle de Données

SQL (relationnel)

User : (id, pseudo[unique], email[unique], password[hash], avatar_url, custom_status, created_at)

Server : (id, name, icon_url, owner_id → User, invite_code[unique], created_at)

Channel : (id, server_id → Server, name, type['text'|'voice'], position, created_at)

Membership : (user_id → User, server_id → Server, role['admin'|'member'], joined_at) // N-N



Relations :

User 1-N Server (un user possède plusieurs serveurs)

Server 1-N Channel (un serveur contient plusieurs salons)

User N-N Server (via Membership, avec un rôle)

MongoDB (documentaire) - collection 'messages'

```
{
  "_id": "ObjectId",
  "channelId": "42",      // salon SQL, ou id de conversation DM
  "scope": "channel",    // "channel" | "dm"
  "authorId": "7",
  "content": "Salut !",
  "attachments": [],     // schéma flexible (bonus : images/fichiers)
  "reactions": [],       // bonus
  "createdAt": "ISODate",
  "editedAt": null
}
```

// Index conseillé : { channelId: 1, createdAt: -1 }

Redis (clé-valeur) -> présence & appels

presence:user:{userId} -> "online" | "away" | "busy" (avec TTL)

online:users -> SET des utilisateurs connectés

voice:channel: {channelId} -> SET des participants du salon vocal



Sécurité

Mesure	Implémentation
Hash des mots de passe	bcrypt avec salt
Authentification	JWT avec expiration
Autorisation WebSocket	Vérification du token à la connexion socket
Autorisation métier	Un membre n'accède qu'à ses serveurs/salons ; seul l'admin gère le serveur
Validation des entrées	Sanitization des messages (anti-XSS)
Requêtes sûres	ORM / requêtes paramétrées (anti-injection)
Secrets	Variables d'environnement, jamais commitées

Organisation de l'Équipe & Gestion de Projet

Vous travaillez en équipe de 3 à 4. La réussite passe autant par le code que par la coordination d'autant que certains membres sont en alternance une partie de la période (voir planning).

Rôles suggérés (à adapter)

Rôle	Responsabilités
Lead / Architecte	Découpage, architecture, intégration, animation du board, garant du jalon
Back temps réel	Express, Socket.io (texte/présence) + signalisation WebRTC
Data & API	Modèles SQL, MongoDB, Redis, sécurité, endpoints



	REST
Front React & WebRTC	UI React, intégration Socket.io client, RTCPeerConnection, UX appel

Méthode imposée

- Board agile (GitHub Projects / Trello / Jira) tenu à jour, tâches reliées aux fonctionnalités.
- GitFlow : branches feature/*, Pull Requests relues entre membres, pas de push direct sur main.
- Daily asynchrone écrit (ex. salon dédié / issue épinglée) : fait / à faire /
- blocages - indispensable pour rester synchro pendant la semaine d'alternance.
- Definition of Done explicite (testé, relu, documenté, mergé).
- Découpage en tâches non-bloquantes pour que les membres absents (alternance) puissent contribuer en autonomie.

Planning sur 2 semaines

Semaine 1 - Fondations & temps réel texte

Jour	Objectif
J1	Cadrage, board, maquettes (Figma), setup mono-repo + Docker Compose (SQL/Mongo/Redis)
J2	Modèle de données SQL (User/Server/Channel/Membership),



	migrations, API Auth (JWT)
J3	Front React : auth, layout serveurs/salons. Back : CRUD serveurs/salons
J4	Socket.io : présence (Redis) + messages de salon temps réel
J5	Historique messages (MongoDB) + DM. Jalon v1-jalon : chat texte complet + conception WebRTC documentée

Semaine 2 – Fondations & temps réel texte

Jour	Objectif
J1	Signalisation WebRTC (offer/answer/ICE) + appel 1-to-1 audio/vidéo
J2	Salon vocal de groupe (mesh ≤ 3) + contrôles (mute/cam/raccrocher)
J3	Indicateurs « en appel », état des appels via Redis, partage d'écran (bonus)
J4	CI/CD (GitHub Actions : lint + tests), finalisation Docker Compose, doc déploiement
J5	Tests de bout en bout, polish UX, README, démo finale

Livrables Attendus

Documents et code attendus dans le dépôt GitHub :

1. Dossier de conception (/docs)



Livrable	Format	Description
Maquettes	Figma/PDF	Écrans principaux (auth, serveur/salon, appel)
MCD/MLD	Image	Modèle relationnel (User/Server/Channel/Membership)
MCD/MLD	Image	Front/Back/SQL/Mongo/Redis + flux WebRTC P2P
Spécification API	Image	Échange de signalisation offer/answer/ICE
Choix techniques	Markdown	Texte, présence et signalisation

2. Application fonctionnelle

Livrable	Description
Code source versionné	Mono-repo Git, historique de commits clairs, PRs
Frontend React TypeScript	SPA fonctionnelle (chat + appels)
Backend API TypeScript	REST + Socket.io (texte/présence/signalisation)
Documentation Swagger	SQL (migrations/seeds) + MongoDB + Redis



Collection Postman	front + back + PostgreSQL/MySQL + MongoDB + Redis
README	Instructions d'installation et lancement

3. Documentation & Tests

Livrable	Description
README.md	Prérequis, installation, docker compose up, lancement
.env.example	Toutes les variables documentées
Plan de tests	Scénarios manuels + tests automatisés
Guide de dépannage	Problèmes WebSocket/WebRTC courants



Pour aller plus loin

- Partage d'écran (getDisplayMedia) fortement valorisé
- Réactions / emojis sur les messages
- Upload de fichiers/images dans le chat (Mongo + stockage)
- Réduction du bruit / flou d'arrière-plan (filtres média)
- Reconnexion automatique WebRTC après coupure réseau
- Nudge / Wizz façon MSN (faire vibrer la fenêtre 😊)
- Thème sombre / clair, notifications navigateur
- Déploiement réel (Render, Railway, VPS) en plus du Compose local

Rendu

Le projet est à rendre sur votre dépôt GitHub :

<https://github.com/prenom-nom/live-chat>

Structure attendue du repository

```
live-chat-discord/
├── README.md
├── docker-compose.yml      # front + back + postgres + mongo + redis
├── .github/
│   └── workflows/
│       └── ci.yml          # lint + tests
├── backend/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       ├── server.js
│       ├── config/         # SQL, Mongo, Redis
│       ├── models/         # Sequelize/Prisma (User, Server, Channel, Membership)
│       ├── repositories/    # accès Mongo (messages), Redis (présence/voix)
│       ├── routes/          # auth, servers, channels
│       ├── middlewares/     # auth JWT
│       └── sockets/
```




```
| | | |— chat.js      # texte + présence
| | | |— webrtc.js   # signalisation
| | | |— utils/
|
|— frontend/
|   |— Dockerfile
|   |— package.json
|   |— index.html
|   |— src/
|       |— main.jsx
|       |— components/    # ChatPanel, ServerBar, ChannelList, CallPanel...
|       |— hooks/         # useSocket, useWebRTC, usePresence
|       |— context/       # auth, socket, call
|       |— pages/
|
|— docs/
|   |— maquettes/
|   |— mcd.png
|   |— architecture.png
|   |— webrtc-sequence.png
|   |— socket-events.md
```

Base de connaissances

WebRTC (la nouveauté à creuser)

- [MDN - WebRTC API](#)
- [MDN - Signaling and video calling](#)
- [MDN - getUserMedia](#)
- [MDN - RTCPeerConnection](#)
- [WebRTC for the Curious \(gratuit\)](#)
- [MDN - Screen Capture API / getDisplayMedia \(partage d'écran\)](#)

WebSockets et [Socket.io](#)

- [Socket.io Documentation](#)
- [Socket.io - Rooms](#)
- [MDN - WebSockets API](#)

React



- [React - Documentation officielle](#)
- [Vite - Guide](#)
- [Zustand / React Context](#)

Bases de données

- [Prisma](#)
- [MongoDB / Mongoose](#)
- [Redis - Documentation / node-redis](#)

Authentification & Sécurité

- [JWT.io](#)
- [jsonwebtoken](#)
- [bcrypt](#)
- [OWASP - XSS Prevention](#)

DevOps

- [Docker Compose](#)
- [GitHub Actions - Quickstart](#)